PRACTICAL – 1

Implementation and Time Analysis of Sorting Algorithms

Objective:

The objective of this practical is to implement and compare the performance of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. We will analyze the time complexity and execution time of each algorithm to understand their efficiency and identify the most suitable algorithm for different applications.

1. Bubble Sort

Code:

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n)
{
    bool swapped;
    for (int i = 0; i < n - 1; i++)
    {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                // Swap adjacent elements
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        // Stop if no swaps occurred
        if (!swapped)
            break;
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    bubbleSort(arr, n);

    cout << "Sorted array: ";
    printArray(arr, n);

    cout << "Roll No: 92460118370" << endl;
    delete[] arr;
    return 0;
}
```

OUTPUT:

```
Enter the number of elements: 5
Enter 5 elements: 8 6 5 7 4
Sorted array: 4 5 6 7 8
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
Best Case	**O(n)**	The array is already sorted. With the optimized version (using the swapped flag), only one pass is required.
Average Case	**O(n²)**	The array is in random order, requiring multiple comparisons and swaps.
Worst Case	**O(n²)**	The array is sorted in reverse order, requiring the maximum number of comparisons and swaps.

Space Complexity: O(1)

2. Selection Sort

Code:

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIdx])
                minIdx = j;
        }
        int temp = arr[i];
        arr[i] = arr[minIdx];
        arr[minIdx] = temp;
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    selectionSort(arr, n);

    cout << "Sorted array: ";
    printArray(arr, n);

    cout << "Roll No: 92460118370" << endl;
    delete[] arr;
    return 0;
}
```

OUTPUT:

```
Enter the number of elements: 5
Enter 5 elements: 64 25 12 22 11
Sorted array: 11 12 22 25 64
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
Best Case	**$O(n^2)$**	Requires n(n-1)/2 comparisons regardless of initial array ordering.
Average Case	**$O(n^2)$**	Scans unsorted subarray for minimum element in each pass.
Worst Case	**$O(n^2)$**	Identical comparison count even for reverse sorted input.

Space Complexity: O(1)

3. Insertion Sort

Code:

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    insertionSort(arr, n);

    cout << "Sorted array: ";
    printArray(arr, n);

    cout << "Roll No: 92460118370" << endl;
    delete[] arr;
    return 0;
}
```

OUTPUT:

```
Enter the number of elements: 5
Enter 5 elements: 12 11 13 5 6
Sorted array: 5 6 11 12 13
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
Best Case	**O(n)**	Array is already sorted; inner loop terminates in 1 comparison per element.
Average Case	**O(n²)**	Requires half array shifts on average per element insertion.
Worst Case	**O(n²)**	Array is reverse sorted; requires n(n-1)/2 element shifts.

Space Complexity: O(1)

4. Merge Sort

Code:

```
#include <iostream>
using namespace std;

void merge(int arr[], int l, int m, int r)
{
    int n1 = m - l + 1;
    int n2 = r - m;
    int* L = new int[n1];
    int* R = new int[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    delete[] L;
    delete[] R;
}

void mergeSort(int arr[], int l, int r)
{
    if (l < r)
    {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main()
{
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    mergeSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;

    cout << "Roll No: 92460118370" << endl;
    delete[] arr;
    return 0;
}
```

OUTPUT:

```
Enter the number of elements: 5
Enter 5 elements: 38 27 43 3 9
Sorted array: 3 9 27 38 43
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
Best Case	**O(n log n)**	Divides array into halves and merges in linear O(n) time.
Average Case	**O(n log n)**	Consistent logarithmic split depth log2(n) across all inputs.
Worst Case	**O(n log n)**	Guaranteed O(n log n) bound solved via Master Theorem.

Space Complexity: O(n)

5. Quick Sort

Code:

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main()
{
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    quickSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;

    cout << "Roll No: 92460118370" << endl;
    delete[] arr;
    return 0;
}
```

OUTPUT:

```
Enter the number of elements: 5
Enter 5 elements: 10 7 8 9 1
Sorted array: 1 7 8 9 10
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
Best Case	**O(n log n)**	Partition splits array evenly into sub-problems of size n/2.
Average Case	**O(n log n)**	Balanced partitioning on random inputs.
Worst Case	**O(n²)**	Pivot selection consistently yields unbalanced partitions.

Space Complexity: O(log n)

PRACTICAL – 2

Implementation and Time Analysis of Searching Algorithms

Objective:

The objective of this practical is to implement and compare the performance of Linear Search and Binary Search algorithms. We will analyze the time complexity and execution steps of each algorithm on unsorted and sorted datasets.

1. Linear Search

Code:

```
#include <iostream>
using namespace std;

int linearSearch(int arr[], int n, int target)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == target)
            return i;
    }
    return -1;
}

int main()
{
    int n, target;
    cout << "Enter number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    cout << "Enter target element to search: ";
    cin >> target;

    int result = linearSearch(arr, n, target);
    if (result != -1)
        cout << "Element found at index: " << result << endl;
    else
        cout << "Element not found" << endl;

    cout << "Roll No: 92460118370" << endl;
    delete[] arr;
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50
Enter target element to search: 30
Element found at index: 2
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
Best Case	**O(1)**	Target element is present at index 0.
Average Case	**O(n)**	Target element is located at index n/2 on average.
Worst Case	**O(n)**	Target element is at index n-1 or completely absent.

Space Complexity: O(1)

2. Binary Search

Code:

```
#include <iostream>
using namespace std;

int binarySearch(int arr[], int n, int target)
{
    int low = 0, high = n - 1;
    while (low <= high)
    {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target)
            return mid;
        if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main()
{
    int n, target;
    cout << "Enter number of elements (sorted): ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " sorted elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    cout << "Enter target element to search: ";
    cin >> target;

    int result = binarySearch(arr, n, target);
    if (result != -1)
        cout << "Target " << target << " found at index: " << result << endl;
    else
        cout << "Target element not found" << endl;

    cout << "Roll No: 92460118370" << endl;
    delete[] arr;
    return 0;
}
```

OUTPUT:

```
Enter number of elements (sorted): 10
Enter 10 sorted elements: 12 25 34 45 50 64 78 88 90 99
Enter target element to search: 78
Target 78 found at index: 6
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
Best Case	**O(1)**	Target element is located at the exact middle index mid.
Average Case	**O(log n)**	Search space is halved at each iteration.
Worst Case	**O(log n)**	Target is at extreme boundary or absent.

Space Complexity: O(1)

PRACTICAL – 3

Implementation of Max Heap Sort

Objective:

The objective of this practical is to implement Max Heap Sort using binary heap data structure operations (heapify and build-max-heap) and analyze its time and space complexity.

1. Max Heap Sort

Code:

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    heapSort(arr, n);

    cout << "Sorted Array (Max Heap Sort): ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;

    cout << "Roll No: 92460118370" << endl;
    delete[] arr;
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 8
Enter 8 elements: 12 11 13 5 6 7 9 20
Sorted Array (Max Heap Sort): 5 6 7 9 11 12 13 20
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
Best Case	**O(n log n)**	Building heap takes O(n), extractions take O(n log n).
Average Case	**O(n log n)**	Guaranteed logarithmic tree height traversal.
Worst Case	**O(n log n)**	Strict upper bound guaranteed across all inputs.

Space Complexity: O(1)

PRACTICAL – 4

Time Analysis of Factorial (Iterative vs Recursive)

Objective:

The objective of this practical is to implement Iterative Factorial and Recursive Factorial in C++ and compare their performance, execution steps, and memory overhead.

1. Factorial Analysis

Code:

```
#include <iostream>
using namespace std;

unsigned long long factorialIterative(int n)
{
    unsigned long long res = 1;
    for (int i = 1; i <= n; i++)
        res *= i;
    return res;
}

unsigned long long factorialRecursive(int n)
{
    if (n <= 1)
        return 1;
    return n * factorialRecursive(n - 1);
}

int main()
{
    int n;
    cout << "Enter a non-negative integer: ";
    cin >> n;

    cout << "Iterative Factorial(" << n << ") = " << factorialIterative(n) << endl;
    cout << "Recursive Factorial(" << n << ") = " << factorialRecursive(n) << endl;

    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

OUTPUT:

```
Enter a non-negative integer: 10
Iterative Factorial(10) = 3628800
Recursive Factorial(10) = 3628800
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
Iterative	**O(n)**	Executes single linear loop with zero call stack overhead.

Recursive	**O(n)**	Allocates n stack frames on call stack.

Space Complexity: Iterative: O(1), Recursive: O(n)

PRACTICAL – 5

Implementation of 0/1 Knapsack using Dynamic Programming

Objective:

The objective of this practical is to solve the 0/1 Knapsack Problem using Dynamic Programming bottom-up tabulation to find maximum profit under capacity constraint W.

1. 0/1 Knapsack DP

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int knapsackDP(int W, int wt[], int val[], int n)
{
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++)
    {
        for (int w = 1; w <= W; w++)
        {
            if (wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }
    return dp[n][W];
}

int main()
{
    int n, W;
    cout << "Enter number of items: ";
    cin >> n;
    int* val = new int[n];
    int* wt = new int[n];
    cout << "Enter values of " << n << " items: ";
    for (int i = 0; i < n; i++) cin >> val[i];
    cout << "Enter weights of " << n << " items: ";
    for (int i = 0; i < n; i++) cin >> wt[i];
    cout << "Enter max capacity W: ";
    cin >> W;

    cout << "Maximum Profit Value in Knapsack = " << knapsackDP(W, wt, val, n) << endl;

    cout << "Roll No: 92460118370" << endl;
    delete[] val;
    delete[] wt;
    return 0;
}
```

OUTPUT:

```
Enter number of items: 3
Enter values of 3 items: 60 100 120
Enter weights of 3 items: 10 20 30
Enter max capacity W: 50
Maximum Profit Value in Knapsack = 220
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
All Cases	**O(n W)**	Fills DP table of size (n+1) x (W+1) using optimal subproblems.

Space Complexity: O(n W)

PRACTICAL – 6

Implementation of Matrix Chain Multiplication

Objective:

The objective of this practical is to implement Matrix Chain Multiplication using Dynamic Programming to determine optimal parenthesization structure minimizing scalar multiplications.

1. Matrix Chain Multiplication

Code:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

int matrixChainOrder(int p[], int n)
{
    vector<vector<int>> m(n, vector<int>(n, 0));

    for (int L = 2; L < n; L++)
    {
        for (int i = 1; i < n - L + 1; i++)
        {
            int j = i + L - 1;
            m[i][j] = INT_MAX;
            for (int k = i; k <= j - 1; k++)
            {
                int q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (q < m[i][j])
                    m[i][j] = q;
            }
        }
    }
    return m[1][n - 1];
}

int main()
{
    int numMatrices;
    cout << "Enter number of matrices: ";
    cin >> numMatrices;
    int* p = new int[numMatrices + 1];
    cout << "Enter dimension array p of size " << numMatrices + 1 << ": ";
    for (int i = 0; i <= numMatrices; i++) cin >> p[i];

    cout << "Minimum Scalar Multiplications Required: " << matrixChainOrder(p, numMatrices + 1) << endl;

    cout << "Roll No: 92460118370" << endl;
    delete[] p;
    return 0;
}
```

OUTPUT:

```
Enter number of matrices: 4
Enter dimension array p of size 5: 10 30 5 60 8
Minimum Scalar Multiplications Required: 4500
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
All Cases	**$O(n^3)$**	Evaluates chain lengths L from 2 to n with split choices k.

Space Complexity: $O(n^2)$

PRACTICAL – 7

Implementation of Coin Change Problem

Objective:

The objective of this practical is to solve the Coin Change Problem (minimum coins required to make amount V) using Dynamic Programming bottom-up tabulation.

1. Coin Change DP

Code:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

int coinChangeMin(int coins[], int n, int amount)
{
    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (i >= coins[j] && dp[i - coins[j]] != INT_MAX)
            {
                dp[i] = min(dp[i], dp[i - coins[j]] + 1);
            }
        }
    }
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}

int main()
{
    int n, amount;
    cout << "Enter number of coin denominations: ";
    cin >> n;
    int* coins = new int[n];
    cout << "Enter " << n << " coin denominations: ";
    for (int i = 0; i < n; i++) cin >> coins[i];
    cout << "Enter target amount: ";
    cin >> amount;

    int ans = coinChangeMin(coins, n, amount);
    if (ans != -1)
        cout << "Minimum coins needed to make amount " << amount << " = " << ans << endl;
    else
        cout << "Amount cannot be formed" << endl;

    cout << "Roll No: 92460118370" << endl;
    delete[] coins;
    return 0;
}
```

OUTPUT:

```
Enter number of coin denominations: 3
Enter 3 coin denominations: 1 2 5
Enter target amount: 11
Minimum coins needed to make amount 11 = 3
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
All Cases	**O(n V)**	Fills 1D array up to target amount V across n coin types.

Space Complexity: O(V)

PRACTICAL – 8

Implementation of Graph Traversals (DFS & BFS)

Objective:

The objective of this practical is to implement Depth-First Search (DFS) and Breadth-First Search (BFS) graph traversal algorithms using adjacency lists.

1. DFS & BFS Graph Traversals

Code:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

void DFS(int node, const vector<vector<int>>& adj, vector<bool>& visited)
{
    visited[node] = true;
    cout << node << " ";
    for (int neighbor : adj[node])
    {
        if (!visited[neighbor])
            DFS(neighbor, adj, visited);
    }
}

void BFS(int startNode, const vector<vector<int>>& adj, int V)
{
    vector<bool> visited(V, false);
    queue<int> q;
    visited[startNode] = true;
    q.push(startNode);

    while (!q.empty())
    {
        int curr = q.front();
        q.pop();
        cout << curr << " ";
        for (int neighbor : adj[curr])
        {
            if (!visited[neighbor])
            {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}

int main()
{
    int V, E;
    cout << "Enter number of vertices and edges: ";
    cin >> V >> E;
    vector<vector<int>> adj(V);
    cout << "Enter " << E << " edges (u v):" << endl;
    for (int i = 0; i < E; i++)
    {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    int startNode = 0;
    cout << "DFS Traversal starting from node 0: ";
    vector<bool> visited(V, false);
    DFS(startNode, adj, visited);
    cout << endl;

    cout << "BFS Traversal starting from node 0: ";
    BFS(startNode, adj, V);
    cout << endl;

    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

OUTPUT:

```
Enter number of vertices and edges: 5 6
Enter 6 edges (u v):
0 1
0 2
1 3
1 4
2 4
3 4
DFS Traversal starting from node 0: 0 1 3 4 2
BFS Traversal starting from node 0: 0 1 2 3 4
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
DFS	**O(V + E)**	Visits every node and edge once in adjacency list.
BFS	**O(V + E)**	Enqueues every node and edge once level-by-level.

Space Complexity: O(V)

PRACTICAL – 9

Implementation of Prim's Algorithm for Minimum Spanning Tree

Objective:

The objective of this practical is to implement Prim's Greedy Algorithm using priority queues to compute the Minimum Spanning Tree (MST) of a weighted connected graph.

1. Prim's Algorithm

Code:

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>
using namespace std;

typedef pair<int, int> pii;

void primsMST(int V, const vector<vector<pii>>& adj)
{
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    vector<int> key(V, INT_MAX);
    vector<int> parent(V, -1);
    vector<bool> inMST(V, false);

    pq.push({0, 0});
    key[0] = 0;
    int totalWeight = 0;

    while (!pq.empty())
    {
        int u = pq.top().second;
        pq.pop();

        if (inMST[u]) continue;
        inMST[u] = true;
        totalWeight += key[u];

        for (auto& edge : adj[u])
        {
            int v = edge.second;
            int weight = edge.first;
            if (!inMST[v] && weight < key[v])
            {
                key[v] = weight;
                pq.push({key[v], v});
                parent[v] = u;
            }
        }
    }

    cout << "Total Minimum Spanning Tree Weight = " << totalWeight << endl;
}

int main()
{
    int V, E;
    cout << "Enter number of vertices and edges: ";
    cin >> V >> E;
    vector<vector<pii>> adj(V);
    cout << "Enter " << E << " edges (u v weight):" << endl;
    for (int i = 0; i < E; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({w, v});
        adj[v].push_back({w, u});
    }

    primsMST(V, adj);

    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

OUTPUT:

```
Enter number of vertices and edges: 5 7
Enter 7 edges (u v weight):
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9
Total Minimum Spanning Tree Weight = 16
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
All Cases	**O((V + E) log V)**	Priority queue heap updates per edge.

Space Complexity: O(V + E)

PRACTICAL – 10

Implementation of Kruskal's Algorithm for MST

Objective:

The objective of this practical is to implement Kruskal's Algorithm for Minimum Spanning Tree using Disjoint Set Union (DSU) with path compression.

1. Kruskal's Algorithm

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Edge {
    int u, v, weight;
    bool operator<(const Edge& other) const {
        return weight < other.weight;
    }
};

class DSU {
    vector<int> parent;
public:
    DSU(int n) : parent(n) {
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]);
    }
    bool unite(int i, int j) {
        int rootI = find(i), rootJ = find(j);
        if (rootI != rootJ) {
            parent[rootJ] = rootI;
            return true;
        }
        return false;
    }
};

void kruskalsMST(int V, vector<Edge>& edges)
{
    sort(edges.begin(), edges.end());
    DSU dsu(V);
    int totalWeight = 0;

    for (const auto& edge : edges)
    {
        if (dsu.unite(edge.u, edge.v))
            totalWeight += edge.weight;
    }

    cout << "Total Minimum Spanning Tree Weight = " << totalWeight << endl;
}

int main()
{
    int V, E;
    cout << "Enter number of vertices and edges: ";
    cin >> V >> E;
    vector<Edge> edges(E);
    cout << "Enter " << E << " edges (u v weight):" << endl;
    for (int i = 0; i < E; i++)
        cin >> edges[i].u >> edges[i].v >> edges[i].weight;

    kruskalsMST(V, edges);

    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

OUTPUT:

```
Enter number of vertices and edges: 4 5
Enter 5 edges (u v weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
Total Minimum Spanning Tree Weight = 19
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
All Cases	**O(E log E)**	Edge sorting dominates complexity.

Space Complexity: O(V + E)

PRACTICAL – 11

Implementation of Floyd-Warshall Algorithm

Objective:

The objective of this practical is to implement the Floyd-Warshall Algorithm for computing All-Pairs Shortest Paths (APSP) using Dynamic Programming matrix operations.

1. Floyd-Warshall Algorithm

Code:

```
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;

const int INF = 99999;

void floydWarshall(int V, vector<vector<int>>& dist)
{
    for (int k = 0; k < V; k++)
    {
        for (int i = 0; i < V; i++)
        {
            for (int j = 0; j < V; j++)
            {
                if (dist[i][k] < INF && dist[k][j] < INF)
                {
                    if (dist[i][k] + dist[k][j] < dist[i][j])
                        dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}

int main()
{
    int V;
    cout << "Enter number of vertices: ";
    cin >> V;
    vector<vector<int>> dist(V, vector<int>(V));
    cout << "Enter adjacency matrix (" << V << "x" << V << ", use 99999 for INF):" << endl;
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            cin >> dist[i][j];

    floydWarshall(V, dist);

    cout << "ALL-PAIRS SHORTEST PATH MATRIX:" << endl;
    for (int i = 0; i < V; i++)
    {
        for (int j = 0; j < V; j++)
        {
            if (dist[i][j] >= INF) cout << setw(8) << "INF";
            else cout << setw(8) << dist[i][j];
        }
        cout << endl;
    }

    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

OUTPUT:

```
Enter number of vertices: 4
Enter adjacency matrix (4x4, use 99999 for INF):
0 5 99999 10
99999 0 3 99999
99999 99999 0 1
99999 99999 99999 0
ALL-PAIRS SHORTEST PATH MATRIX:
       0       5       8       9
     INF       0       3       4
     INF     INF       0       1
     INF     INF     INF       0
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
All Cases	**$O(V^3)$**	Three nested loops over V vertices.

Space Complexity: $O(V^2)$

PRACTICAL – 12

Implementation of Travelling Salesman Problem

Objective:

The objective of this practical is to solve the Travelling Salesman Problem (TSP) using Held-Karp Bitmask Dynamic Programming to find minimum tour cost visiting all cities once.

1. Travelling Salesman Problem (Held-Karp DP)

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

const int INF = 1e9;

int tsp(int mask, int pos, int N, const vector<vector<int>>& dist, vector<vector<int>>& dp)
{
    if (mask == (1 << N) - 1)
        return dist[pos][0];

    if (dp[mask][pos] != -1)
        return dp[mask][pos];

    int ans = INF;
    for (int city = 0; city < N; city++)
    {
        if ((mask & (1 << city)) == 0)
        {
            int newCost = dist[pos][city] + tsp(mask | (1 << city), city, N, dist, dp);
            ans = min(ans, newCost);
        }
    }
    return dp[mask][pos] = ans;
}

int main()
{
    int N;
    cout << "Enter number of cities: ";
    cin >> N;
    vector<vector<int>> dist(N, vector<int>(N));
    cout << "Enter " << N << "x" << N << " distance matrix:" << endl;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            cin >> dist[i][j];

    vector<vector<int>> dp(1 << N, vector<int>(N, -1));
    int minCost = tsp(1, 0, N, dist, dp);

    cout << "Minimum Cost to Visit All Cities and Return to Start = " << minCost << endl;

    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

OUTPUT:

```
Enter number of cities: 4
Enter 4x4 distance matrix:
0 20 42 25
20 0 30 34
42 30 0 12
25 34 12 0
Minimum Cost to Visit All Cities and Return to Start = 80
Roll No: 92460118370

...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Case	Time Complexity	Description
All Cases	**O(n² 2■)**	State space of 2■ masks x n cities.

Space Complexity: O(n 2■)